AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

Mechatronics and Computer Programs
Spring 2026

CSE323: Advanced Embedded Systems Design
CSE411/CSE411s: Real-Time and Embedded Systems Design

Assignment (4)

Objective

This assignment aims to develop a **deep understanding of how FreeRTOS performs context switching**, with a special focus on the macro that **forces a context switch from within an ISR**.

By the end of this task, you should be able to:

- Explain how context switching works internally in FreeRTOS.
- Analyze the role of specific macros used in interrupt handling.
- Demonstrate the effect of forced context switching in a real embedded system.

Background

In a real-time operating system like FreeRTOS, **context switching** is the mechanism that allows the CPU to switch from one task to another.
Normally, switching occurs:

- At **tick interrupts**
- When a task **blocks or yields**

However, in interrupt-driven systems, an ISR may wake up a higher-priority task, requiring an immediate context switch.

This is achieved using a critical macro:

portYIELD_FROM_ISR(xHigherPriorityTaskWoken);

📌 Task Description

You are required to perform a **detailed technical analysis and practical validation** of the-FreeRTOS macro responsible for forcing a context switch.

🔍 Part 1: Theoretical Analysis

✔ Task 1.1: Identify the Macro

Locate and study the macro: **portYIELD_FROM_ISR( )**

📍 You should:

- Find its definition in the FreeRTOS port layer (for ARM Cortex-M)
- Trace what it expands to (e.g., assembly instructions or function calls)

✔ Task 1.2: Explain Its Function

Provide a clear explanation of:

- What does “yield” mean in RTOS?
- Why is a special macro needed inside ISRs?
- What is the role of: **xHigherPriorityTaskWoken**

✔ Task 1.3: Deep Dive (Very Important)

Analyze what happens internally when this macro is executed:

💡 You are expected to explain:

- How the PendSV interrupt is used
- Why PendSV is chosen for context switching
- How interrupt priorities affect this mechanism

👉 Keywords you should research:

- PendSV handler
- NVIC interrupt priorities
- Cortex-M exception model

Part 2: Practical Implementation

✔ Task 2.1: Create a Simple System

Develop a FreeRTOS application with:

- Two tasks:
  - Task A (low priority)
  - Task B (high priority)
- One interrupt source:
  - Timer OR GPIO

✔ Task 2.2: ISR Behavior

Inside the ISR:

1. Trigger an event:
   - Give semaphore OR
   - Send notification
2. Use:
   BaseType_t xHigherPriorityTaskWoken = pdFALSE;
3. Call:
   portYIELD_FROM_ISR(xHigherPriorityTaskWoken);

✔ Task 2.3: Observation

You must demonstrate:

- Without portYIELD_FROM_ISR():
  - Task switch happens later (on next tick)
- With portYIELD_FROM_ISR():
  - Task switch happens immediately after ISR

📊 Part 3: Experimental Analysis

You are required to compare both cases:

Case	Behavior
Without macro	Delayed response
With macro	Immediate response

✔ Required Measurements

You should measure or observe:

- Task execution order.
- Latency between interruption and task execution.

💡 Suggested tools:

- LED toggling.
- UART timestamps.
- Debugger breakpoints.

📄 Part 4: Short Report Requirements (2-3 pages)

Your report must include:

1. Macro Analysis

- Full explanation of: **portYIELD_FROM_ISR( )**
- Internal mechanism (PendSV, NVIC, etc.)

2. Code Discussion

- Key parts of your implementation
- ISR design

3. Experimental Results

- Observations (with/without macro)
- Timing or behavioral differences

⚠️ Important Notes

- You must not perform heavy processing inside ISR
- Only use FromISR APIs:
  - **xSemaphoreGiveFromISR()**
  - **xQueueSendFromISR()**
- Ensure proper interrupt priority configuration: **NVIC_SetPriority(...)**

💡 Hint

Check these FreeRTOS files:

- port.c
- portmacro.h